

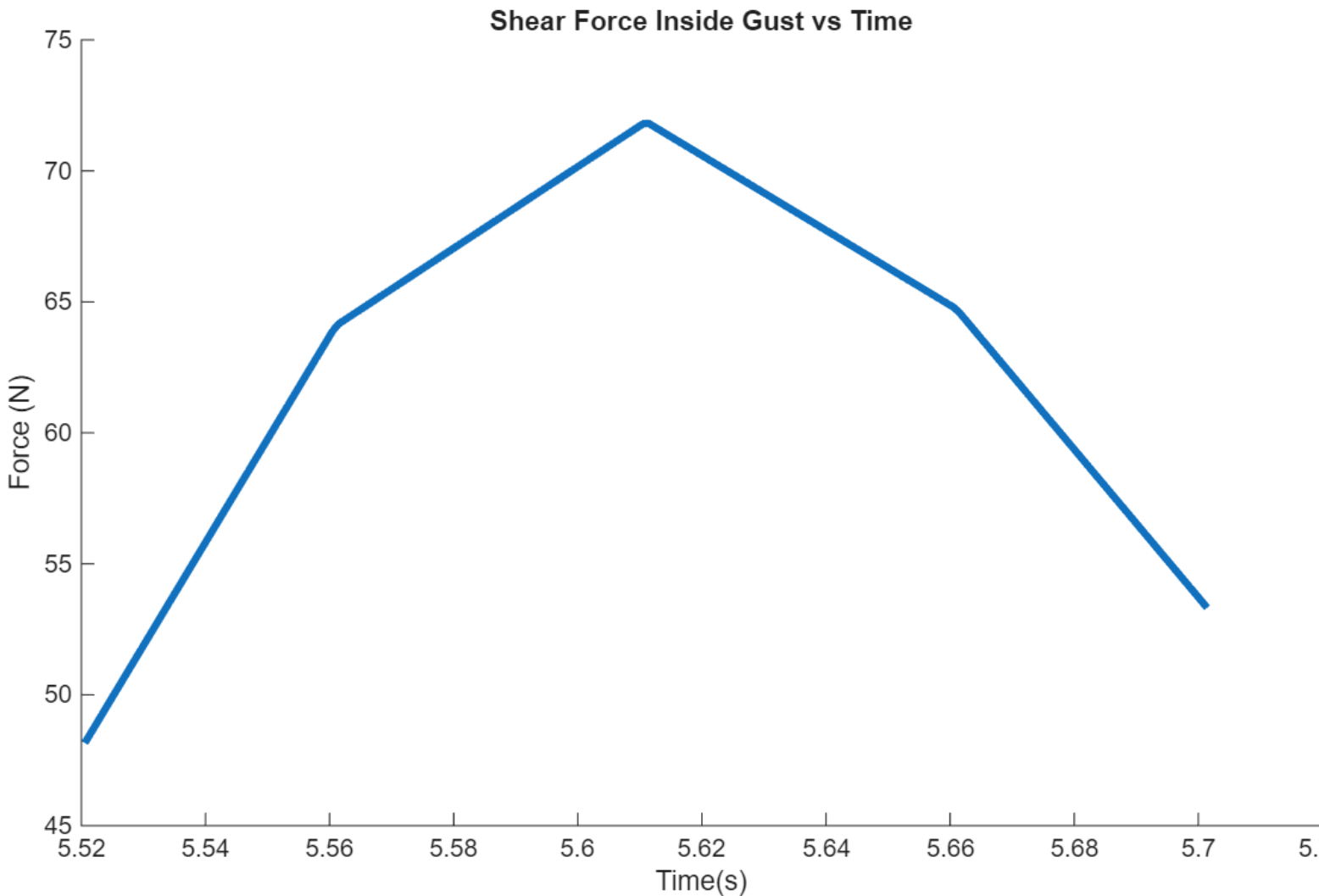
Dynamic Bending Test Rig

12.500 Hz is the required actuator bandwidth, and the required force is 16.8787 lbf as per section 2.2 . Assuming that you are interested in replicating the force (rather than displacement)

Potentially look at decreasing required actuator bandwidth by smoothing out the function? So ignoring the decrease that occurs between 5.5 and 5.7

1 Actuator Sizing from Shear Forces MATLAB (as of 3/6/26)

Now, I have a force/time graph from the previous detailed shear force analysis.



But we are unsure about the validity of this. A way to confirm the validity would be to compare it to a 'vector projection' of the max shear to the angle of attack, using small angle approximation

Quickest ramp looks to be

```
timeForceRamp = 5.56-5.52
```

```
timeForceRamp =  
0.0400
```

```
freqTimeRamp = 1/timeForceRamp
```

```
freqTimeRamp =  
25.0000
```

```
deltaForce = 63 - 55
```

```
deltaForce =  
8
```

2.1 Actuator Sizing from Gust Velocity Profile & Ian's Calculations

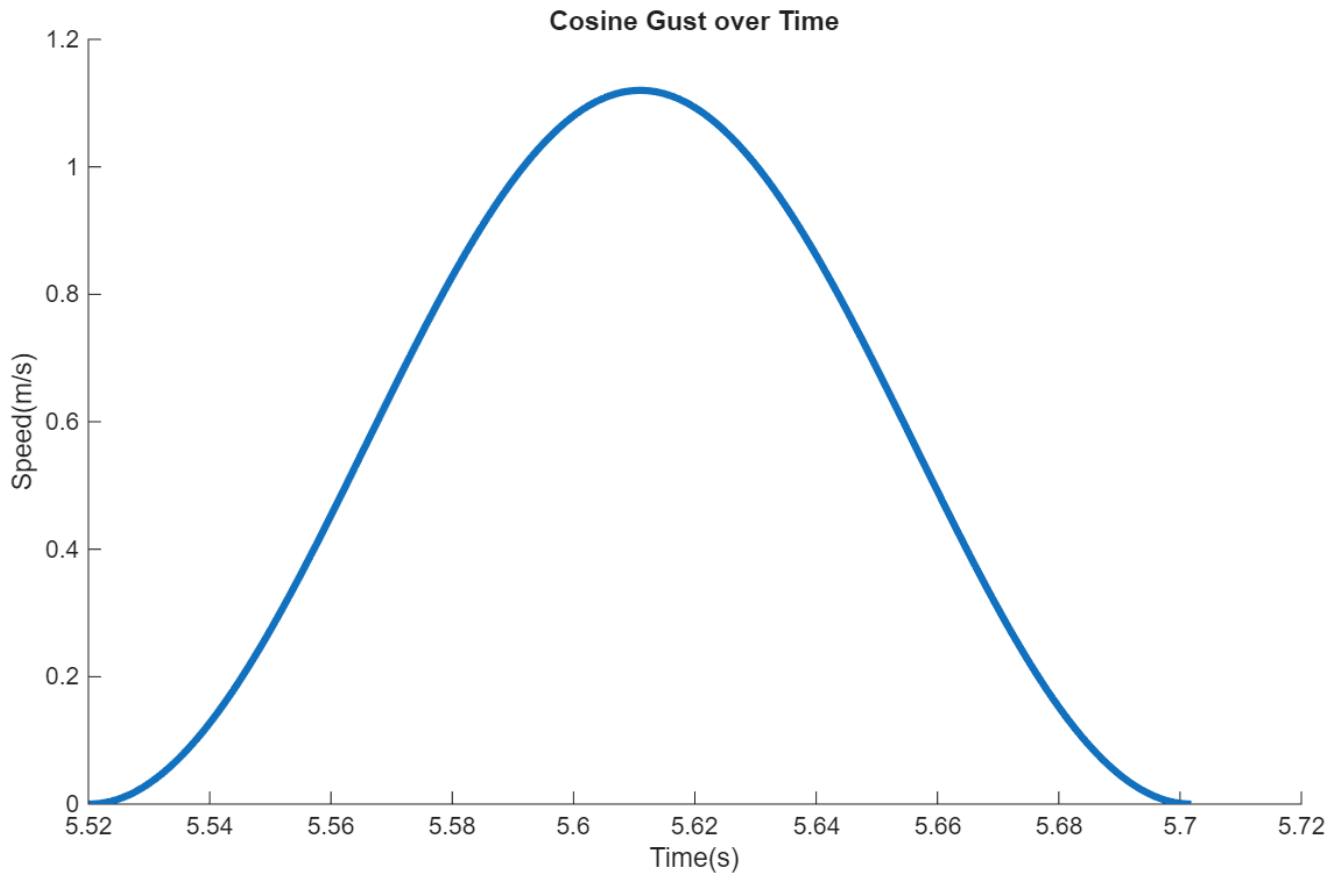
From Ian's Shear Calculations (at 1.7 AoA):

https://docs.google.com/spreadsheets/d/1YpCOBGylhGbyl9DqY5GLDXi1CcHhjao9K5FwuqBK1_Q/edit?usp=sharing

		2.744895879	1.18593478	-63.05414586	86.91810389
m5 inertial load	2.75	-91.44732192	-154.5014678	86.12950966	
	2.820098507	0.07498686142	-154.4264809	75.30444392	
	2.895301135	0.4359773457	-153.9905036	63.72395334	
	2.970503763	0.4987899236	-153.4917136	52.18097307	
	3.045706391	0.5478739359	-152.9438397	40.67919436	
	3.12090902	0.6578691834	-152.2859705	29.22688914	
	3.196111648	6.472251167	-145.8137194	18.26131422	
	3.271314276	56.66729428	-89.14642508	11.55726876	
	3.346516904	106.8224795	17.67605444	12.88655451	
	3.421719532	146.274175	163.9502294	25.21604265	
m6 inertial load	3.43	-274.9276715	-110.9774421	24.29709753	
	3.49692216	73.6050986	-37.37234349	21.79605956	
	3.572124789	21.27224074	-16.10010275	20.58528952	
	3.647327417	-2.184223439	-18.28432619	19.21026014	
	3.722530045	44.74627641	26.46195022	21.20026834	
m7 inertial load	3.83	-26.45747998	0.004470241051	21.20074876	

164 N is the max shear, which is about 36.8687 lbf

From MATLAB code for the Cosine Gust:



The time this occurs at is around 5.52s to 5.7s, which results in a 0.18s time for the gust

Apply a factor of safety of two, so that there is a 0.09s cycling requirement for the cosine gust.

This results in the required bandwidth being as follows(from here) https://en.wikipedia.org/wiki/Amplifier_figures_of_merit?:

$$f = \frac{3}{.09}$$

fBW1 = 3/.09 % in Hz

fBW1 =
33.3333

2.2 Gust Shear Profile Based Off OpenRocket AoA & Ian's Calculations

Take the AoA profile from OpenRocket and multiply it by Ian's peak shear at peak angle of attack to transform the AoA curve to a shear curve

```
%% OpenRocket CSV -> Angle of Attack vs Time & Altitude (robust)
% - Handles OpenRocket comment lines (# ...)
% - Uses the detected header line for variable names
% - Robustly finds Time / Altitude / AoA columns
% - Forces vectors to be column vectors to avoid table row-size errors

clear; clc;

filePath = "OpenRocket.csv"; % or full path if needed

%% 1) Find the header line + first data line (first non-# line)
fid = fopen(filePath,'r');
assert(fid ~= -1, "Could not open file: %s", filePath);

headerLine = "";
dataStartLine = NaN;

lineNum = 0;
while ~feof(fid)
    lineNum = lineNum + 1;
    ln = fgetl(fid);
    if ~ischar(ln), break; end
    lnTrim = strtrim(ln);

    % OpenRocket commonly writes: "# Time (s),Altitude (m),..."
    if startsWith(lnTrim, "#") && contains(lnTrim, "Time") && contains(lnTrim,
"Altitude")
        headerLine = erase(lnTrim, "#");
        headerLine = strtrim(headerLine);
    end

    % First actual data row is typically the first non-# line
    if ~startsWith(lnTrim, "#") && ~isempty(lnTrim) && isnan(dataStartLine)
        dataStartLine = lineNum;
        break;
    end
end
fclose(fid);
```

```

assert(~isempty(headerLine), "Couldn't find the OpenRocket header row (the '#
Time...,Altitude...' line).");
assert(~isnan(dataStartLine), "Couldn't find the first data row (first non-#
line).");

%% 2) Build MATLAB-friendly variable names from the header
rawNames = split(headerLine, ",");
rawNames = strtrim(rawNames);

% makeValidName removes spaces/symbols so e.g. "Time (s)" -> "Times"
varNames = matlab.lang.makeValidName(rawNames, "ReplacementStyle","delete");

%% 3) Read numeric data from the first data line onward
opts = delimitedTextImportOptions("NumVariables", numel(varNames));
opts.DataLines = [dataStartLine, Inf];
opts.Delimiter = ",";
opts.VariableNames = varNames;
opts.VariableTypes = repmat("double", 1, numel(varNames));
opts.ExtraColumnsRule = "ignore";
opts.EmptyLineRule = "read";

T = readtable(filePath, opts);

%% 4) Find the Time / Altitude / AoA columns (works with unit-suffixed names)
names = string(T.Properties.VariableNames);

% exact-match candidates (common OpenRocket outputs after makeValidName)
timeCandidates = ["Times","Time_s","Time","time","TimeSeconds"];
altCandidates = ["Altitudem","Altitude_m","Altitude","altitude"];
aoaCandidates =
["AngleOfAttack","AngleOfAttack_deg","Angle_of_attack","AoA","alpha"];

timeVar = intersect(timeCandidates, names, "stable");
altVar = intersect(altCandidates, names, "stable");
aoaVar = intersect(aoaCandidates, names, "stable");

% fallback: contains (case-insensitive)
if isempty(timeVar)
    timeVar = names(contains(lower(names), "time"));
end
if isempty(altVar)
    altVar = names(contains(lower(names), "altitude"));
end
if isempty(aoaVar)
    aoaVar = names(contains(lower(names), "angleofattack") | contains(lower(names),
"aoa"));
end

assert(~isempty(timeVar), "Couldn't find Time column. Columns found: %s",
strjoin(names, ", "));

```

```

assert(~isempty(altVar), "Couldn't find Altitude column. Columns found: %s",
strjoin(names, ", "));
assert(~isempty(aoaVar), "Couldn't find Angle of attack column. Columns found:
%s", strjoin(names, ", "));

% pull data
t = T.(timeVar(1));
alt = T.(altVar(1));
aoa = T.(aoaVar(1));

%% 5) Force all to be column vectors (fixes 'same number of rows' table error)
t = t(:);
alt = alt(:);
aoa = aoa(:);

% assert(numel(t)==numel(alt) && numel(t)==numel(aoa), ...
% "Lengths mismatch: t=%d, alt=%d, aoa=%d", numel(t), numel(alt), numel(aoa));
%
% AoA_Time_Altitude = table(t, alt, aoa, ...
% "VariableNames", ["Time_s", "Altitude_m", "AngleOfAttack_deg"]);
%
% % 6) Display + plots
% disp("Detected columns:");
% disp(" Time: " + timeVar(1));
% disp(" Altitude: " + altVar(1));
% disp(" AoA: " + aoaVar(1));
%
% disp(AoA_Time_Altitude(1:min(10,height(AoA_Time_Altitude)), :));
%
% figure;
% plot(AoA_Time_Altitude.Time_s, AoA_Time_Altitude.AngleOfAttack_deg);
% xlabel("Time (s)"); ylabel("Angle of attack (deg)");
% grid on; title("Angle of Attack vs Time");
%
% figure;
% scatter(AoA_Time_Altitude.Altitude_m, AoA_Time_Altitude.AngleOfAttack_deg, 10,
"filled");
% xlabel("Altitude (m)"); ylabel("Angle of attack (deg)");
% grid on; title("Angle of Attack vs Altitude");

```

Then to generate the graph for shear (from the max shear calculated via CFD at 1.7 AoA):

164 N ~ 36.8687 lbf

```
shearMax = 36.8687
```

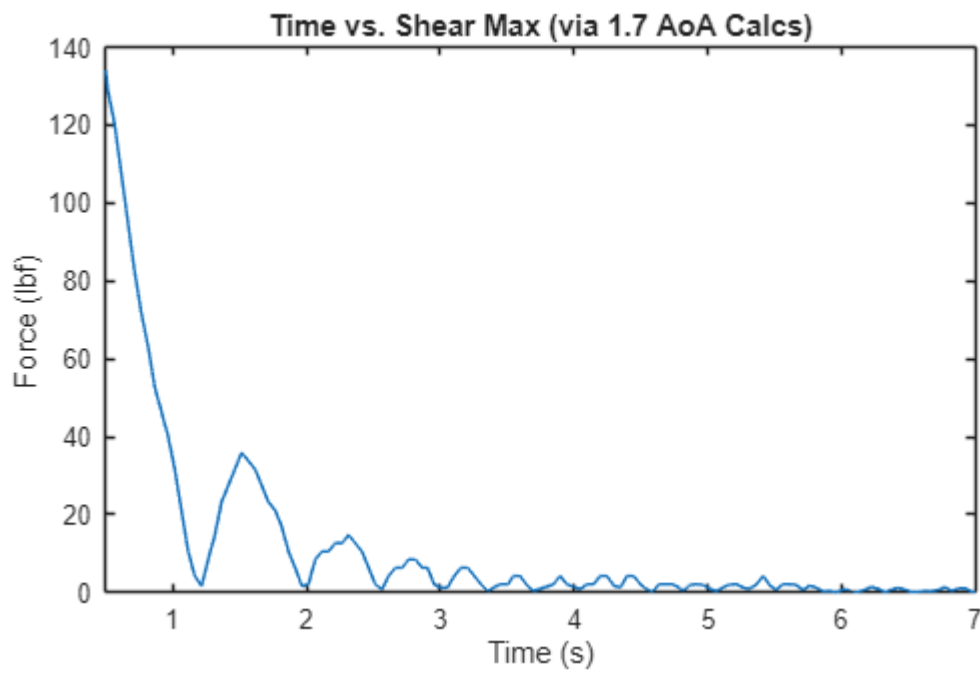
```
shearMax =
36.8687
```

```
shearGraph = aoa*shearMax
```

```
shearGraph = 1244x1
```

```
103 ×  
3.3186  
3.3165  
3.2489  
3.0736  
2.8306  
2.4694  
NaN  
2.0681  
1.7153  
1.4449  
1.2379  
1.0752  
0.9464  
0.8408  
0.7541  
⋮
```

```
figure  
plot(t,shearGraph)  
xlim([0.5, 7])  
title("Time vs. Shear Max (via 1.7 AoA Calcs)")  
xlabel("Time (s)"); ylabel("Force (lbf)")
```



Look at the 6.6 AoA

	2.74489588	5.7427833	-195.2332689	315.8000347
m5 inertial load	2.745	-332.3405003	-527.5737692	315.7451037
	2.82009851	2.66213073	-524.9116384	276.3250218
	2.89530114	2.83878607	-522.0728524	237.0637703
	2.97050376	3.18801036	-518.884842	198.0422707
	3.04570639	2.85598701	-516.028855	159.2355436
	3.12090902	4.55424976	-511.4746052	120.7713081
	3.19611165	19.2521009	-492.2225043	83.75488125
	3.27131428	174.474026	-317.7484783	59.85936
	3.3465169	346.922944	29.17446566	62.05335626
	3.42171953	482.646504	511.8209697	100.5436393
m6 inertial load	3.4345	-1004.016361	-492.1953916	94.25315083
	3.49692216	373.629416	-118.5659756	86.85200653
	3.57212479	131.519018	12.95304235	87.82610938
	3.64732742	19.3586972	32.31173955	90.25603717
	3.72253005	69.7030941	102.0148337	97.92782096
m7 inertial load	3.833	-96.78717484	5.22765881	98.50532017

524.9116 N ~ 118.00482242 lbf

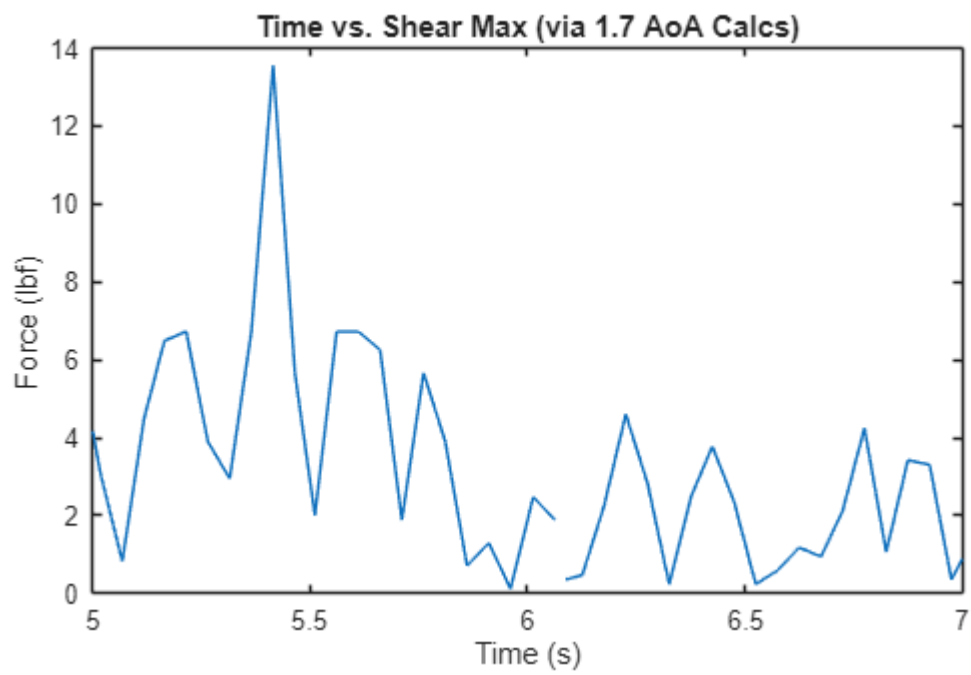
```
shearMax = 118.00482242
```

```
shearMax =  
118.0048
```

```
shearGraph = aoa*shearMax
```

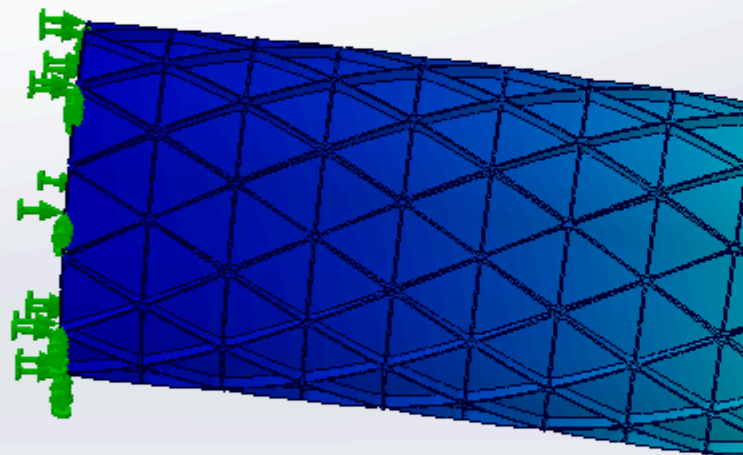
```
shearGraph = 1244x1  
104 ×  
1.0622  
1.0615  
1.0399  
0.9837  
0.9060  
0.7904  
NaN  
0.6619  
0.5490  
0.4625  
0.3962  
0.3441  
0.3029  
0.2691  
0.2414  
⋮  
⋮
```

```
figure  
plot(t,shearGraph)  
xlim([5, 7])  
title("Time vs. Shear Max (via 1.7 AoA Calcs)")  
xlabel("Time (s)"); ylabel("Force (lbf)")
```



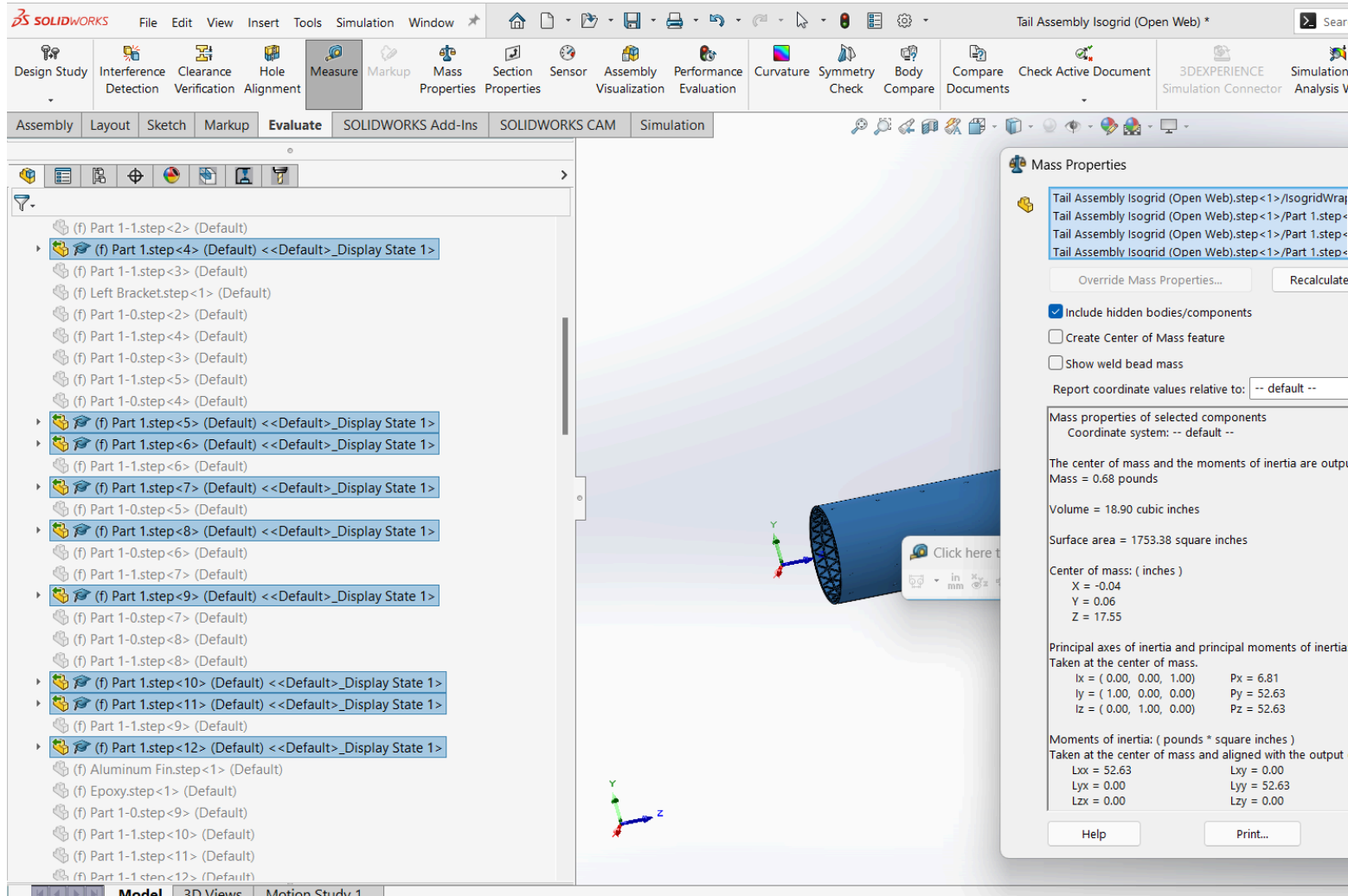
This amount of force looks to be too conservative. Did FEA for Stroke Length

Model name: TAI
Study name: Static 1(-Default-)
Plot type: Static displacement Displacement1
Deformation scale: 807.298



Therefore, stroke length = 9.13373×10^{-5} m for the solid isogrid.

For the open web...



Volume = 18.90 inches

Let's look at 1.7 AoA and get the limits to get a better look at the quickest ramp required:

$$\text{shearMax} = 36.8687$$

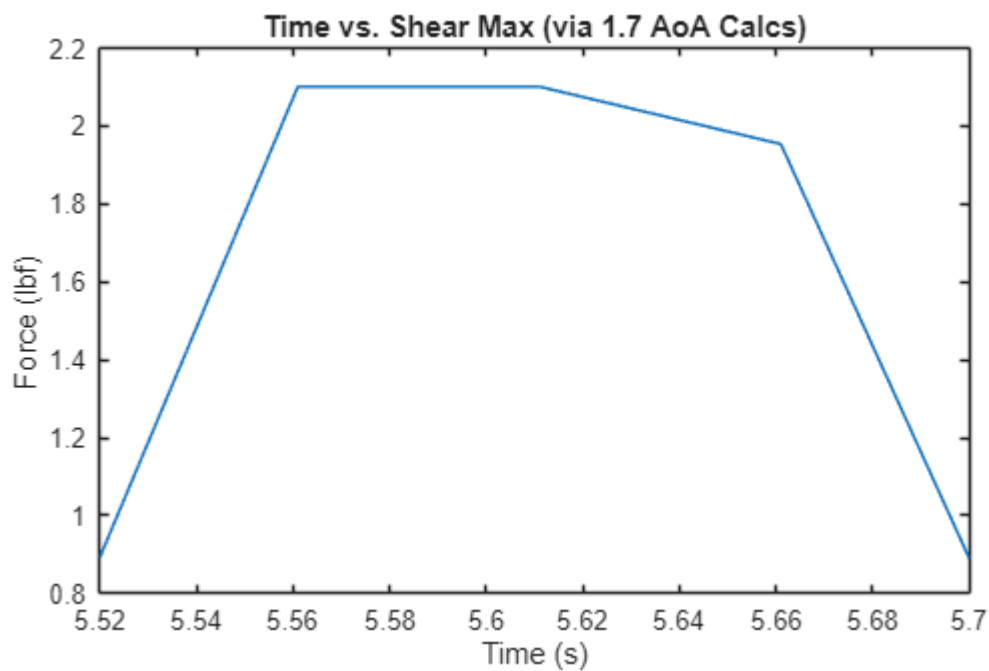
$$\text{shearMax} = 36.8687$$

$$\text{shearGraph} = \text{aoa} * \text{shearMax}$$

$$\text{shearGraph} = 1244 \times 10^3 \times$$

3.3186
 3.3165
 3.2489
 3.0736
 2.8306
 2.4694
 NaN
 2.0681
 1.7153
 1.4449
 1.2379
 1.0752
 0.9464
 0.8408
 0.7541
 ⋮

```
figure
plot(t,shearGraph)
xlim([5.52 5.7])
title("Time vs. Shear Max (via 1.7 AoA Calcs)")
xlabel("Time (s)"); ylabel("Force (lbf)")
```



2.3 Sizing Actuator Bandwidth and Required Force, Based Off OpenRocket AoA & Ian's Calculations

Time is at 5.64 to 5.72~

0.08s required for the cycle, and then bandwidth would be

$$FBW2 = 1 / .08$$

$$FBW2 = 12.5000$$

12.500 Hz is the required actuator bandwidth, and the required force is 16.8787 lbf

These calculations are more conservative than the previous guesstimate, so use this instead. The 2x required force from the 1.7 AoA is used in the above conclusion

But what if I removed that drop that occurs at 5.65 seconds? then I would be looking at 5.700 - 5.525

$$FBW2smoothed = 1 / (5.700 - 5.525)$$

$$FBW2smoothed = 5.7143$$

2.3 Calculate Displacement Over Time Due to Shear

Please see the PDF below for an initial displacement estimation:

Thin Walled Cylinder Approximation for Deflection

From Onshape CAD

$$R = 3.02"$$

$$r = 3"$$

$$\text{length} = 29.996"$$

$$R_{twc} = 3.020$$

$$R_{twc} = 3.0200$$

$$r_{twc} = 3$$

$$r_{twc} = 3$$

$$h_{twc} = 29.996$$

$$h_{twc} = 29.9960$$

```
volTWC = pi*(Rtwc^2-rtwc^2)*htwc
```

```
volTWC =  
11.3459
```

```
% from Shigley's, table A-5 for aluminum alloy  
rho6061 = 0.098 % lbf/in^3
```

```
rho6061 =  
0.0980
```

```
% from Shigley's, table A-5 for aluminum alloy  
E6061 = 10.4*10^6 % psi
```

```
E6061 =  
10400000
```

```
% calc mass  
massTWC = rho6061*volTWC
```

```
massTWC =  
1.1119
```

```
% need to load the shear somehow from the .mat, as well as the time  
% from the prev. detailed analysis we did  
shearDGustProfile = load('shearDGustProfile.mat');
```

```
% import time data for shear  
shearDGustTimeDis = load('shearDGustTimeDist.mat')
```

```
shearDGustTimeDis = struct with fields:  
gustArray: [200x4 double]
```

```
Ptwc = shearDGustProfile.totalShearFnOfGustVel(:,1)
```

```
Ptwc = 200x1  
48.1568  
48.5147  
48.8727  
49.2306  
49.5886  
49.9465  
50.3045  
50.6624  
51.0204  
51.3783  
51.7363  
52.0942  
52.4522  
52.8101
```

53.1681

⋮

```
shearTimeArr = shearDGustTimeDis.gustArray(:,1)
```

```
shearTimeArr = 200×1
```

5.5206

5.5215

5.5224

5.5233

5.5242

5.5251

5.5260

5.5269

5.5278

5.5287

5.5296

5.5306

5.5315

5.5324

5.5333

⋮

```
% conversion factor is for newtons to lbs (1/4.448)
```

```
Ptwc = Ptwc/4.448
```

```
Ptwc = 200×1
```

10.8266

10.9071

10.9876

11.0680

11.1485

11.2290

11.3095

11.3899

11.4704

11.5509

11.6314

11.7118

11.7923

11.8728

11.9533

⋮

```
% area moment of inertia for tube
```

```
Dtwc = Rtwc*2
```

```
Dtwc =
```

6.0400

```
dtwc = rtwc*2
```

```
dtwc =
```

6

$$aITWC = ((Dtwc^4 - dtwc^4)*\pi)/64$$

$$aITWC = 1.7135$$

% now that we have all the information for thin wall deflection...
% calculate it!:

$$defBendTWC = (Ptwc.*(htwc^3))/(3*E6061*aITWC)$$

$$\begin{aligned} defBendTWC &= 200 \times 1 \\ &0.0055 \\ &0.0055 \\ &0.0055 \\ &0.0056 \\ &0.0056 \\ &0.0057 \\ &0.0057 \\ &0.0058 \\ &0.0058 \\ &0.0058 \\ &0.0059 \\ &0.0059 \\ &0.0060 \\ &0.0060 \\ &0.0060 \\ &\vdots \\ &\vdots \end{aligned}$$

Recall : $V \propto A \propto I$

Use this relationship to estimate a ratio between the $I_{isogrid}$ and I_{TWC}

From previous calculations (units are in³).

Where V_{twc} is manually calculated, and V_{iso} is recorded from onshape

$$\frac{V_{iso}}{V_{TWC}} = \frac{25.029}{11.3491} = 2.2059$$

$$Recall : \delta_b = \frac{PL^3}{3EI}$$

Then, using the ratio and proportionalities established above:

$$\delta_{b,isogrid} = \frac{1}{2.2059} \delta_{b,TWC}$$

$$\delta_{b,isogrid} = 0.4533 * \delta_{b,TWC}$$

Using this ratio, then plug-in numbers for the calculated shear so you can be a force vs. displacement profile to match with actuators.

Note that from FEA there is an error of 6.8982%. This could be because the arc rings are not included in the CAD for the FEA, which could have reduced expected length. Still good nonetheless.

```
% user inputted from previous analyses:
```

```
twcTOISOFACOR = 2.2059
```

```
twcTOISOFACOR =
```

```
2.2059
```

```
defBendISOEst = defBendTWC/twcTOISOFACOR
```

```
defBendISOEst = 200x1
```

```
0.0025
```

```
0.0025
```

```
0.0025
```

```
0.0025
```

```
0.0026
```

```
0.0026
```

```
0.0026
```

```
0.0026
```

```
0.0026
```

```
0.0026
```

```
0.0027
```

```
0.0027
```

```
0.0027
```

```
0.0027
```

```
0.0027
```

```
:
```

```
:
```

```
% now graph force/displacement
```

```
Piso = Ptwc
```

```
Piso = 200x1
```

```
10.8266
```

```
10.9071
```

```
10.9876
```

```
11.0680
```

```
11.1485
```

```
11.2290
```

```
11.3095
```

```
11.3899
```

```
11.4704
```

```
11.5509
```

```
11.6314
```

```
11.7118
```

```
11.7923
```

```
11.8728
```

```
11.9533
```

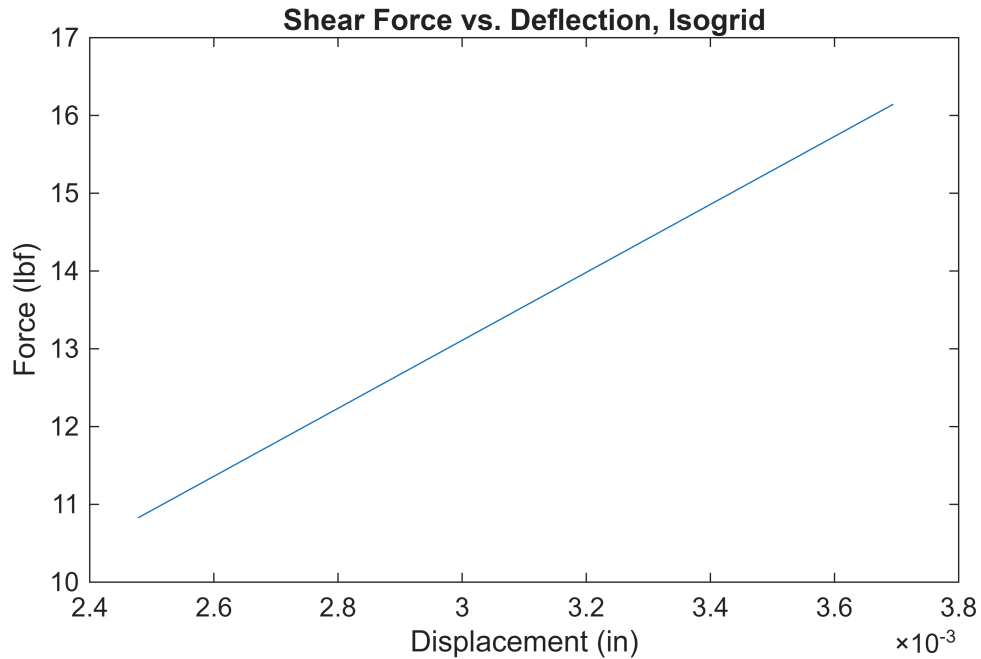
```
:
```

```
:
```

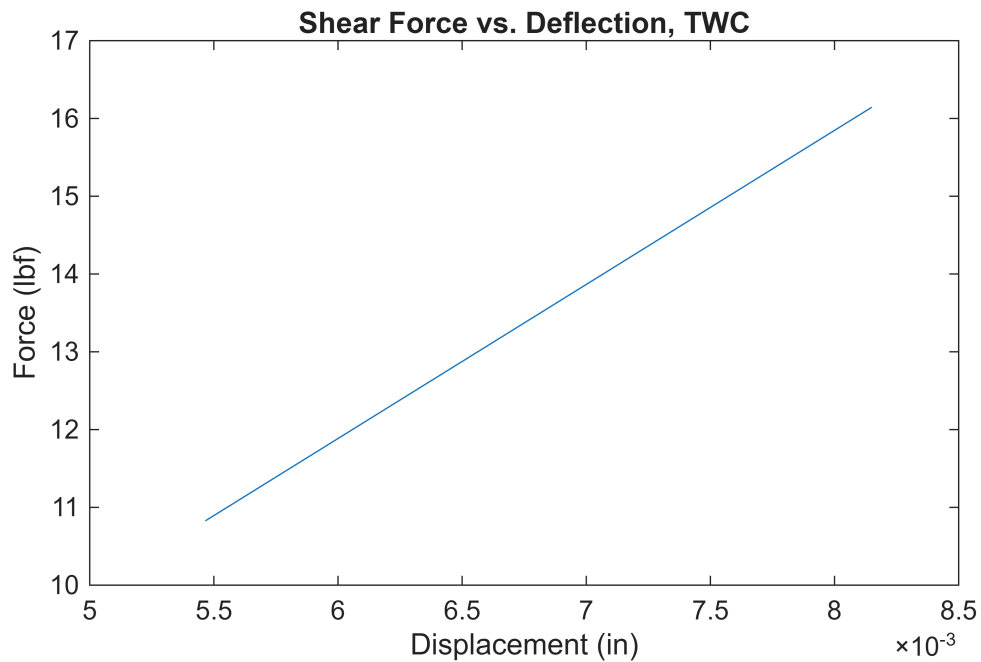
```
figure;
```

```
plot(defBendISOEst, Piso)
```

```
title('Shear Force vs. Deflection, Isogrid'); xlabel('Displacement (in)');
ylabel('Force (lbf)')
```



```
figure;
plot(defBendTWC, Ptwc)
title('Shear Force vs. Deflection, TWC'); xlabel('Displacement (in)');
ylabel('Force (lbf)')
```

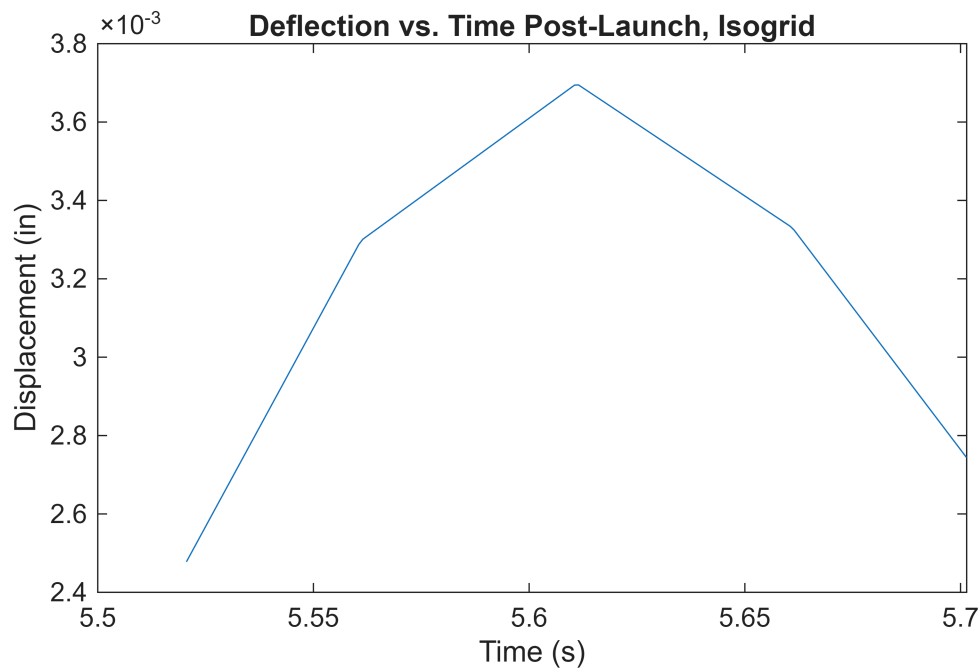


```
% new graph, displacement over time
```

```

plot(shearTimeArr, defBendISOEst)
title('Deflection vs. Time Post-Launch, Isogrid'); xlabel('Time (s)');
ylabel('Displacement (in)')

```

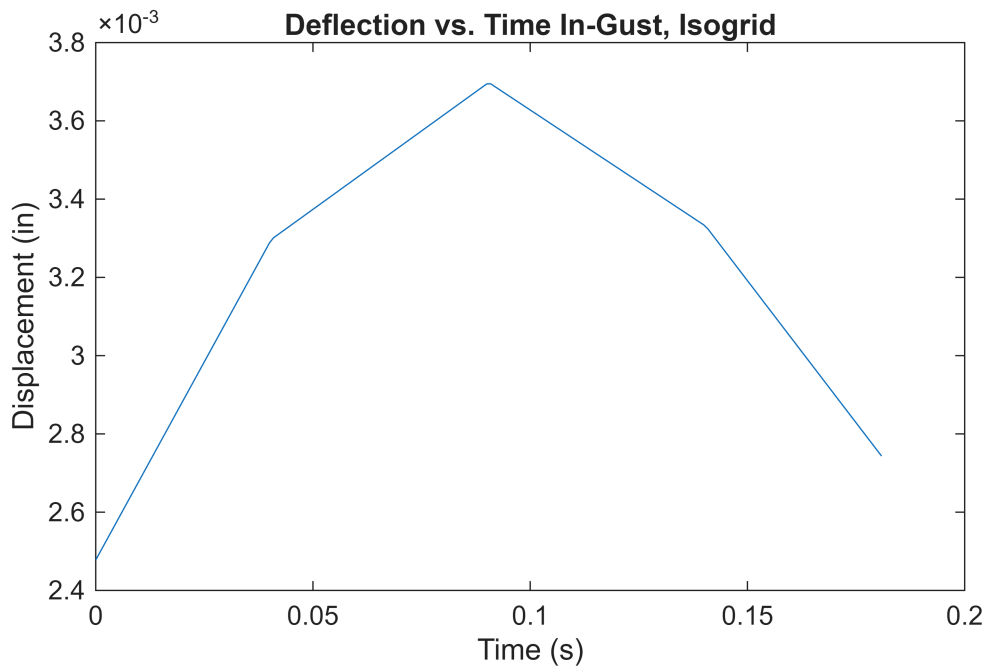


```

% new graph, time shifted for 0 - whatever seconds
min_valTime = min(shearTimeArr);
shearTimeArr0Shift = shearTimeArr - min_valTime;

plot(shearTimeArr0Shift, defBendISOEst)
title('Deflection vs. Time In-Gust, Isogrid'); xlabel('Time (s)');
ylabel('Displacement (in)')

```



2.4 Calculate Required Acceleration from Graph

3.1 Actuator Choice: Calculating how actuator maps to speed requirements (if we were interested in displacement)

A consideration I had was whether or not we wanted to replicate the displacement caused by the shear forces. To analyze what kind of actuator we would need if we wanted to replicate displacement (as well as force application), I applied calculations that assumed simple harmonic movement of a linear actuator.

	2.744885079	1.18083478	-0.300414080	80.91810389
m5 inertial load	2.75	-01.44732192	-154.5014678	86.12950966
	2.820098507	0.07498686142	-154.4264809	75.30444392
	2.895301135	0.4359773457	-153.9905036	63.72395334
	2.970503763	0.4987899236	-153.4917136	52.18097307
	3.045706391	0.5478739359	-152.9438397	40.67919436
	3.12090902	0.6578691834	-152.2859705	29.22688914
	3.196111648	6.472251167	-145.8137194	18.26131422
	3.271314276	56.66729428	-89.14642508	11.55726876
	3.346516904	106.8224795	17.67605444	12.88655451
	3.421719532	146.274175	163.9502294	25.21604265
m6 inertial load	3.43	-274.9276715	-110.9774421	24.29709753
	3.49682216	73.6050988	-37.37234349	21.79605956
	3.572124789	21.27224074	-16.10010275	20.58528952
	3.647327417	-2.184223439	-18.28432619	19.21026014
	3.722530045	44.74627641	26.46195022	21.20026834
m7 inertial load	3.83	-26.45747998	0.004470241051	21.20074876

From the figure above(Ian's Calculations), grab the lever arm as the length from the nose cone to the point of max shear

```
leverArm = 3.417 % in m
```

```
leverArm =  
3.4170
```

Using the radius, and recalling that for SHM:

X(t) =

```
% Used FEA to get the displacement below  
strokeLength = 9.13373*10^-5 % in m
```

```
strokeLength =  
9.1337e-05
```

```
% peak velocity  
velPeakMS = 2*pi * FBW2*strokeLength % in m/s
```

```
velPeakMS =  
0.0072
```

```
velPeakmillMS = velPeakMS*100
```

```
velPeakmillMS =  
0.7174
```

```
velPeakSmoothed = 2*pi*FBW2smoothed*strokeLength
```

```
velPeakSmoothed =  
0.0033
```

$$V_{max} = 2\pi fA$$

% peak acceleration

accPeak = ((2*pi * FBW2)^2)*strokeLength% in m/s

accPeak =
0.5634

These values indicate that looking to replicate the displacement as well as the force profile would prove expensive, due to it requiring a peak velocity at 46.8395 m/s , and a peak acceleration of

Required Rise Time:

0.08s for the

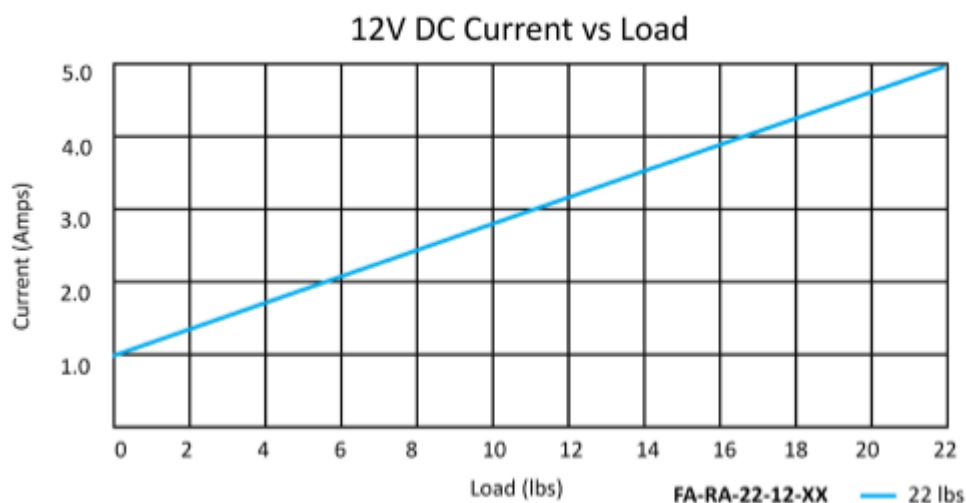
This guesstimation of a stroke length of 0.2982 seems to be too large. Let's sanity check that

3.2.1 Options for Actuators that Meet Requirements, HS Electric LA

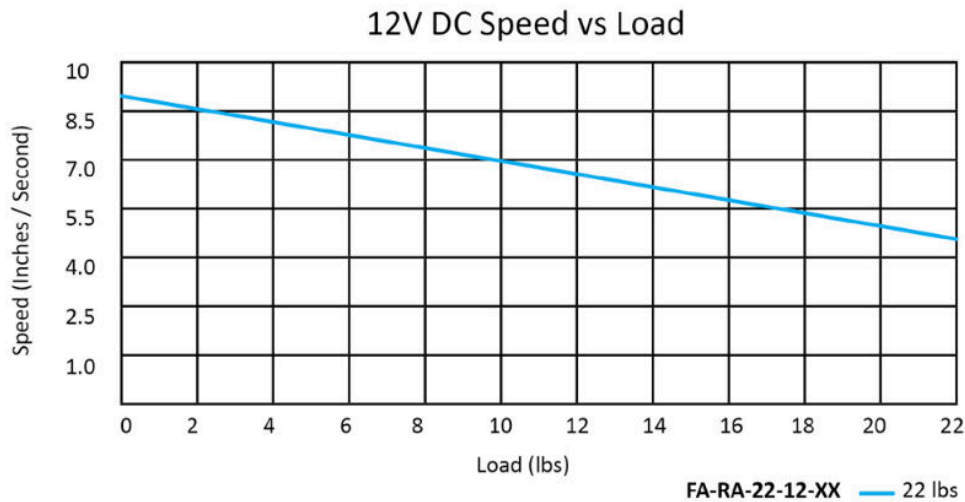
Firgelli High Speed Linear Actuator

<https://cdn-firgelliauto.sfo2.cdn.digitaloceanspaces.com/product-downloads/linear-actuator/firgelli-high-speed-linear-actuator-spec-sheet.pdf>

Look at the force requirement



Look at velocity(bandwidth) requirement



Does not meet speed requirement. (7 in/s at 8 lbf) $v_{la} = 0.185$ m/s . This is way under velocity peak of 46.8395

% look at the speed requirement

$v_{la} = 0.185$

$v_{la} =$
0.1850

$\text{freqEstLA} = v_{la} / (2 \cdot \pi \cdot \text{strokeLength})$ % in Hz

$\text{freqEstLA} =$
322.3619

Waay too slow

3.2.2 Options for Acutators that Meet Requirements, Voice Coil LA

[NCC15-24-090-1X](#)

Non-Commutated DC Linear Actuator. Moving Coil Type. 9.0 lbs (40 N) Continuous Force, 27 lbs (120 N) Peak Force @ 10% duty, 1.5" (38.1 mm) Travel. OD = 2.38" (60.5 mm), Length at mid-stroke = 3.13" (79.5 mm), radial clearance on either side of coil = 0.063" (0.38 mm). Reference H2W drawing # 30-0984 Rev -.

Please note: This motor requires a customer-supplied bearing system for it to operate properly.

Price: 1 piece @ US\$1,983 each

NON-COMM ACTUATOR SPECIFICATIONS		
Motor P/N	NCC15-24-090-1X	
Device Type	Moving Coil	
Stroke	1.50 in	38.1 mm
Radial Clearance	0.015 in	0.38 mm
Bearing Type	None Provided	
Moving Mass	0.75 lbs	340 grams
Total Mass	2.38 lbs	1080 grams
Resistance @ 20C	5.0 ohms	
Inductance @ 20C	2.8 mH	
Electrical Time Constant	0.56 msec	
Motor Constant	1.82 lbs/sqrt(watt)	8.1 N/sqrt(watt)
Force Constant	4.1 lbs/amp	18.1 N/amp
Back EMF	0.46 V/ips	18.1 V/m/sec
Force @ 100% Duty	9.0 lbs	40 N
Power @ 100% Duty	30 watts	
Current @ 100% Duty	2.2 amps	
Force @ 10% Duty	27 lbs	120 N
Power @ 10% Duty	220 watts	
Current @ 10% Duty	6.6 amps	

Looking at Force Requirements (for specified displacement)

```
forceConstant = 4.1;
% calculate required amperage for force requirement
forceReq2 = 16.8787;
```

```
forceReq1 = forceReq2/2
```

```
forceReq1 =
8.4393
```

```
ampReqForce2 = forceConstant*forceReq2
```

```
ampReqForce2 =
69.2027
```

```
ampReqForce1 = forceConstant*forceReq1
```

```
ampReqForce1 =
34.6013
```

```
% assume 10% duty... 220 watts to determine required voltage
```

$$P = IV$$

$$V = P/I$$

$$V_{ReqForce2} = 220/ampReqForce2$$

$$V_{ReqForce2} = 3.1791$$

$$V_{ReqForce1} = 220/ampReqForce1$$

$$V_{ReqForce1} = 6.3581$$

$$P_{ReqForce2} = V_{ReqForce2} * ampReqForce2$$

$$P_{ReqForce2} = 220$$

$$P_{ReqForce1} = V_{ReqForce1} * ampReqForce1$$

$$P_{ReqForce1} = 220$$

Additionally, at 10% duty meets the force requirement as 27lbs > 16.67 lbf

Requires a PSU with > 220 Watts, and the Vin ~= 3.2V

Looking at Peak Velocity Requirements (for specified displacement corresponding to Bandwidth via SHM)

Back EMF is noted to be 0.46 V/ips, and also 18.1 V/m/sec

```
% note that vel peak is in meters/s  
BEMF = 18.1;  
reqVfromBEMF = (velPeakMS*BEMF) % in volts
```

$$reqVfromBEMF = 0.1298$$

$$reqVfromBEMFSmooth = (velPeakSmoothed*BEMF)$$

```
reqVfromBEMFSmooth =
0.0594
```

```
% do delta V
```

```
% look at the force constant
```

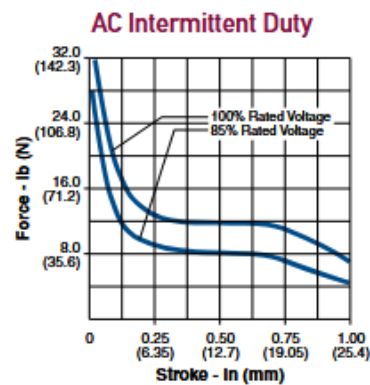
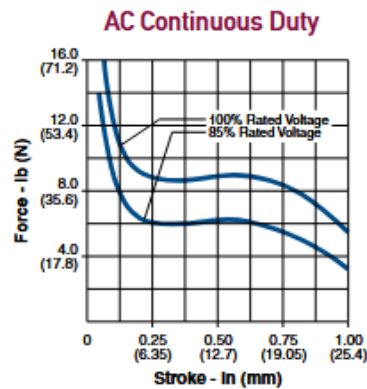
```
% calculate wattage requirement
```

```
powerReqVCLA = ampReqForce1*reqVfromBEMF + PReqForce2
```

```
powerReqVCLA =
224.4927
```

```
% 29 kW requirement
```

3.2.3 Solenoid Actuator



Duty Cycle	Continuous		Intermittent	
Model	2536-F-34	2537-F-34	2786-F-34	2787-F-34
Volts — 60Hz (50 Hz avail.)	120	240	120	240
Coil Resistance $\pm 10\%$ (Ohms at 25°C)	14.8	60	11.7	48
Watts Seated $\pm 10\%$	19.0	18.0	36.5	36.0
Amps Seated $\pm 10\%$	0.48	0.24	0.95	0.48
Amps at $\frac{1}{4}'' \pm 10\%$	2.90	1.50	4.30	2.20
Amps at $\frac{1}{2}'' \pm 10\%$	4.40	2.20	6.00	3.00
Amps at $\frac{3}{4}'' \pm 10\%$	5.50	2.70	7.50	3.80
Amps at $1'' \pm 10\%$	6.50	3.20	8.50	4.30

https://www.johnsonelectric.com/pub/media/image/tmp/metric-imperial/Dormeyer_AC_Laminate_3.pdf

Looking at intermittent Dormeyer 2786-F-34 Solenoid.

```
strokesInsideQuarterInch = 0.25./defBendISOEst; % qty to help calculate amperage
required
```

```
quarterInchAmperage = 4.30 % user inputted from solenoid data above
```

```
quarterInchAmperage =  
4.3000
```

```
solenoidAmperage = quarterInchAmperage.*strokesInsideQuarterInch;
```

```
% since it is a 120VAC solenoid...
```

```
wattageSolenoid = 120*solenoidAmperage
```

```
wattageSolenoid = 200×1
```

```
104 ×
```

```
5.2063  
5.1679  
5.1301  
5.0928  
5.0560  
5.0198  
4.9841  
4.9488  
4.9141  
4.8799  
4.8461  
4.8128  
4.7800  
4.7476  
4.7156  
:  
:
```

```
plot(shearTimeArr0Shift, wattageSolenoid)  
title('Power COnsumption vs. Time In-Gust, Isogrid'); xlabel('Time (s)');  
ylabel('Power Consumption(W)')
```

